NumPy Quick Reference

Beginner-friendly revision sheet based on the class notebooks numpy-1 and numpy2

How to use this sheet

- Run the examples **in order, from top to bottom**. Later examples reuse variables created earlier (for example `arr1`, `arr2` and `arr3` from Section 2).
- Random values change every run. Outputs marked **Sample output** will look different on your computer.
- Lines that cause an error are wrapped in `try / except`, so the example still runs and prints the error message. In class, the same lines stopped the notebook with a red error.
- Boxes: **Note** = extra information, **Common mistake** = something to avoid, **Correction** = something fixed from the class notebooks.
- Outputs were produced with NumPy 2.5 and Python 3.12. The default integer type may print as `int32` instead of `int64` on some computers. This is normal.

Contents

1. Introduction: Lists vs NumPy Arrays
2. Creating an ndarray with np.array()
3. Array Attributes
4. Data Types (dtype)
5. Creating Arrays from Existing Data (Bonus)
6. Creating Arrays from Scratch
7. Creating Arrays with Random Values
8. Creating Arrays with Range Functions
9. Special Matrices for Linear Algebra (Bonus)
10. Indexing and Slicing
11. Advanced Indexing
12. Iterating over an Array
13. Reshaping and Flattening
14. Joining, Transposing and Splitting
15. Arithmetic Operators and Mathematical Functions
16. Logical Functions
17. Sorting
18. Searching
19. Counting in an ndarray
20. Statistical Functions
21. Linear Algebra

1. Introduction: Lists vs NumPy Arrays

NumPy is a Python library for fast number crunching. Its main object is the **ndarray** (n-dimensional array): a grid of items that all have the same data type.

1.1 Importing NumPy

Import NumPy once at the top of your file. `np` is the standard short name.

SYNTAX
```
import numpy as np
```

```
import numpy as np
```

1.2 List vs ndarray: multiplying by a number

A Python list repeats when you multiply it. A NumPy array multiplies **every item** instead. This is called vectorization: it needs no loop and is faster for large data.

```
arr = [1, 2, 3]
print(arr * 5)                 # list: repeats the list 5 times

arr = [x * 5 for x in arr]     # a list needs a list comprehension (or loop)
print(arr)
```

OUTPUT

```
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
[5, 10, 15]
```

```
arr1 = np.array([1, 2, 3])
arr1 = arr1 * 5                # ndarray: every item is multiplied
print(arr1)
```

OUTPUT

```
[ 5 10 15]
```

2. Creating an ndarray with np.array()

Pass a list (or a list of lists) to `np.array()`. Each extra level of square brackets adds one dimension: 1D is a row of values, 2D is a table of rows and columns, 3D is a stack of tables.

SYNTAX

`np.array(object, dtype=None)`

```
# 1D array
arr1 = np.array([1, 2, 3, 4, 5])

# 2D array (2 rows, 3 columns)
arr2 = np.array([[1, 2, 3],      # row 1
                 [4, 5, 6]])     # row 2

# 3D array (3 "floors", each floor has 2 rows and 3 columns)
arr3 = np.array([
    # 1st floor
    [[1, 2, 3],
     [4, 5, 6]],
    # 2nd floor
    [[1, 2, 3],
     [4, 5, 6]],
    # 3rd floor
    [[1, 2, 3],
     [4, 5, 6]]
])

print(arr1)
print(arr2)
```

OUTPUT

```
[1 2 3 4 5]
[[1 2 3]
 [4 5 6]]
```

Note: `arr1`, `arr2` and `arr3` are used again in many later sections.

3. Array Attributes

Attributes describe an array. They are written **without brackets**.

Attribute	Meaning	Example result for arr2
`ndim`	Number of dimensions	2
`shape`	Size of each dimension (a tuple)	(2, 3)
`dtype`	Data type of the items	int64
`size`	Total number of items	6

```
# dimension
print(arr1.ndim)
print(arr2.ndim)
print(arr3.ndim)

# shape
print(arr1.shape)
print(arr2.shape)
print(arr3.shape)

# data type
print(arr1.dtype)
print(arr2.dtype)
print(arr3.dtype)

# size
print(arr1.size)
print(arr2.size)
print(arr3.size)
```

```
OUTPUT
1
2
3
(5,)
(2, 3)
(3, 2, 3)
int64
int64
int64
5
6
18
```

Note: `arr3.shape` is `(3, 2, 3)`: 3 floors, 2 rows, 3 columns. `size` is the product of the shape: 3 × 2 × 3 = 18. A 1D shape such as `(5,)` has a comma because it is a tuple with one item.

4. Data Types (dtype)

4.1 Default data type

NumPy chooses the type from your data: whole numbers become `int64`, decimals become `float64`.

```
arr = np.array([1, 2, 3])
print(arr.dtype)

arr = np.array([1.5, 2.4, 3.2])
print(arr.dtype)
```

OUTPUT
```
int64
float64
```

4.2 Mixed types: upcasting

An array can hold only one data type. If you mix types, NumPy converts everything to the widest type: **int / bool → float → string**.

```
arr = np.array([1, 2, 3.2])
print(arr.dtype)               # upcasted to float

arr = np.array([1, 2, 3.2, 'hello'])
print(arr.dtype)               # upcasted to string
print(arr)

arr = np.array([1, True, 3.25, 'hello'])
print(arr.dtype)
```

OUTPUT
```
float64
<U32
['1' '2' '3.2' 'hello']
<U32
```

Note: `<U32` means a Unicode string of up to 32 characters. Strings cannot be used in maths, so avoid mixing text with numbers.

4.3 Choosing a data type: dtype and astype()

Use `dtype=` when creating an array, or `astype()` to convert an existing array. `astype()` returns a **new** array, so assign the result back.

SYNTAX
```
np.array(data, dtype=np.int16)
arr.astype(np.int32)
```

```
arr = np.array([1, 2, 300], dtype=np.int16)
print(arr.dtype)
print(arr)

arr = arr.astype(np.int32)
print(arr.dtype)
```

OUTPUT
```
int16
[  1   2 300]
int32
```

Common mistake: The data cannot always be converted. A string such as `'hello'` cannot become a float, so NumPy raises a `ValueError`.

```
try:
    arr = np.array([1, 2, 3.2, 'hello'], dtype=np.float64)
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
ValueError: could not convert string to float: 'hello'
```

Note: Smaller types save memory but hold smaller numbers. For example, `int16` can only store values from -32768 to 32767.

5. Creating Arrays from Existing Data (Bonus)

`np.array()` accepts lists and tuples directly. For a set or the parts of a dictionary, convert to a list first with `list()`.

5.1 From a list

```
lst = [10, 20, 30, 40, 40.5]
arr = np.array(lst, dtype=np.int32)
print(arr)                    # 40.5 is cut to 40

mixed_lst = [10, True, 'Hello']
arr = np.array(mixed_lst)
print(arr)
print(arr.dtype)

matrix = [[1, 2, 3],          # row 1
          [4, 5, 6]]          # row 2
arr = np.array(matrix)
print(arr)
print(type(arr))
print(arr.dtype)
print(arr.shape)
print(arr.ndim)
```

OUTPUT

```
[10 20 30 40 40]
['10' 'True' 'Hello']
<U21
[[1 2 3]
 [4 5 6]]
<class 'numpy.ndarray'>
int64
(2, 3)
2
```

Note: Converting decimals to an integer type **cuts off** the decimal part (40.5 becomes 40). It does not round.

5.2 From a tuple

```
tpl = (10, 20, 30)

arr = np.array(tpl, dtype=np.int32)
print(arr.dtype)

arr = arr.astype(np.int16)
print(arr)
print(type(arr))
print(arr.dtype)
print(arr.shape)
print(arr.ndim)
```

```
OUTPUT
int32
[10 20 30]
<class 'numpy.ndarray'>
int16
(3,)
1
```

5.3 From a set

```
st = {1, 2, 3}

arr = np.array(list(st))       # convert the set to a list first
arr = arr.astype(np.int16)
print(arr)
print(type(arr))
print(arr.dtype)
print(arr.shape)
print(arr.ndim)
```

```
OUTPUT
[1 2 3]
<class 'numpy.ndarray'>
int16
(3,)
1
```

Common mistake: `np.array(st)` on a set does not make a normal array. It makes a 0-dimensional array of type `object`. Always use `np.array(list(st))`. Also, a set has no fixed order, so the item order may differ.

```
bad = np.array(st)
print(bad.ndim, bad.dtype)
```

```
OUTPUT
0 object
```

5.4 From a dictionary

A dictionary gives three views: `keys()`, `values()` and `items()`. Convert each one with `list()` before using `np.array()`.

```
dc = {'a': 10, 'b': 20, 'c': 30}

keys = dc.keys()
values = dc.values()
items = dc.items()
```

```
# keys to ndarray
arr = np.array(list(keys))
print(arr)
print(arr.dtype)
print(arr.shape)
```

OUTPUT

```
['a' 'b' 'c']
<U1
(3,)
```

```
# values to ndarray
arr = np.array(list(values))
print(arr)
arr = arr.astype(np.int16)
print(arr.dtype)
print(arr.shape)
```

OUTPUT

```
[10 20 30]
int16
(3,)
```

```
# items to ndarray
arr = np.array(list(items))
print(arr)
print(arr.dtype)
print(arr.shape)
print(arr.ndim)
print(arr.size)
```

OUTPUT

```
[['a' '10']
 ['b' '20']
 ['c' '30']]
<U21
(3, 2)
2
6
```

Note: `items()` gives (key, value) pairs, so the result is 2D with shape `(3, 2)`. The numbers become strings because the keys are strings.

6. Creating Arrays from Scratch

These functions build an array from a **shape** (a tuple such as `(2, 3)`). Each has a `_like` version that copies the shape of an existing array, for example `arr3`.

Function	What it creates
`np.zeros`	All items are 0
`np.ones`	All items are 1
`np.empty`	Items are not set (random leftover memory values)
`np.full`	All items are the value you choose

6.1 np.zeros and np.zeros_like

SYNTAX

```
np.zeros(shape, dtype=float)
np.zeros_like(a, dtype=None)
```

```
arr = np.zeros((2, 3), dtype=np.int8)
print(arr)

arr = np.zeros_like(arr3)      # same shape as arr3
print(arr.shape)
print(arr.dtype)
```

OUTPUT

```
[[0 0 0]
 [0 0 0]]
(3, 2, 3)
int64
```

6.2 np.ones and np.ones_like

SYNTAX

```
np.ones(shape, dtype=float)
np.ones_like(a, dtype=None)
```

```
arr = np.ones((3, 4, 3), dtype=np.int8)
print(arr.shape)

arr = np.ones_like(arr3)       # same shape as arr3
print(arr.shape)
print(arr[0])                  # the first "floor"
```

OUTPUT

```
(3, 4, 3)
(3, 2, 3)
[[1 1 1]
 [1 1 1]]
```

6.3 np.empty and np.empty_like

SYNTAX

```
np.empty(shape, dtype=float)
np.empty_like(a, dtype=None)
```

```
arr = np.empty((4, 3), dtype=np.int8)
print(arr.shape)

arr = np.empty_like(arr3)      # same shape as arr3
print(arr.shape)
```

OUTPUT

```
(4, 3)
(3, 2, 3)
```

Correction from class notes: In class, `np.empty_like(arr3)` printed all 1s. That was only leftover memory. `np.empty` does **not** set any values, so never rely on them. Use it only when you will fill the array yourself afterwards (it is slightly faster than `zeros`).

6.4 np.full and np.full_like

Creates an array filled with only one value.

SYNTAX

```
np.full(shape, fill_value, dtype=None)
np.full_like(a, fill_value, dtype=None)
```

```
arr = np.full((4, 3), np.inf)
print(arr)

arr = np.full_like(arr3, np.inf, dtype=np.float64)
print(arr.shape)
print(arr.dtype)

print(np.full((2, 3), 7))      # any value can be used
```

OUTPUT

```
[[inf inf inf]
 [inf inf inf]
 [inf inf inf]
 [inf inf inf]]
(3, 2, 3)
float64
[[7 7 7]
 [7 7 7]]
```

Note: `np.inf` is a float, so `full_like(arr3, np.inf)` needs `dtype=np.float64`. Without it, NumPy keeps the integer type of `arr3` and raises an error.

7. Creating Arrays with Random Values

Function	Gives	Range
`np.random.rand(rows, cols)`	Random decimals	0 up to (not including) 1
`np.random.randint(start, end, shape)`	Random whole numbers	start up to (not including) end
`np.random.uniform(start, end, shape)`	Random decimals	start up to end

```
# np.random.rand(rows, cols): decimals between 0 and 1
arr = np.random.rand(2, 3)

print(arr)
print(type(arr))
print(arr.dtype)
print(arr.shape)
print(arr.ndim)
print(arr.size)
```

SAMPLE OUTPUT (values differ every run)

```
[[0.37454012 0.95071431 0.73199394]
 [0.59865848 0.15601864 0.15599452]]
<class 'numpy.ndarray'>
float64
(2, 3)
2
6
```

```python
# np.random.randint(start, end, shape): end is NOT included
arr = np.random.randint(1, 10, (2, 3))

print(arr)
print(arr.dtype)
print(arr.shape)
```

SAMPLE OUTPUT (values differ every run)

```
[[8 5 4]
 [8 8 3]]
int64
(2, 3)
```

```python
# np.random.uniform(start, end, shape): decimals between start and end
arr = np.random.uniform(50, 100, (3, 3, 4))

print(arr.dtype)
print(arr.shape)
print(arr.ndim)
print(arr.size)
```

OUTPUT

```
float64
(3, 3, 4)
3
36
```

Note: `rand` takes the sizes as separate numbers, `rand(2, 3)`. `randint` and `uniform` take the shape as a tuple, `(2, 3)`. The dtype of `randint` may be `int32` or `int64`, depending on your computer.

Common mistake: `np.random.rand((2, 3))` (with a tuple) raises a `TypeError`. Write `np.random.rand(2, 3)`.

8. Creating Arrays with Range Functions

8.1 np.arange

Works like Python's `range()`, but returns an array. The end value is **not** included.

SYNTAX

```
np.arange(start, stop, step)
```

```
arr = np.arange(1, 10, 1)
mat = arr.reshape(3, 3)

print(arr)
print(mat)
print(type(arr))
print(arr.dtype)
print(arr.shape)
print(arr.ndim)
print(arr.size)
```

OUTPUT

```
[1 2 3 4 5 6 7 8 9]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
<class 'numpy.ndarray'>
int64
(9,)
1
9
```

8.2 np.linspace

Gives a fixed **number of points** spread evenly between start and end. The end value **is** included.

SYNTAX

```
np.linspace(start, stop, num)
```

```
arr = np.linspace(0, 4, 6)     # 6 points from 0 to 4

print(arr)
print(arr.dtype)
print(arr.shape)
```

OUTPUT

```
[0.  0.8 1.6 2.4 3.2 4. ]
float64
(6,)
```

8.3 np.logspace

Like `linspace`, but `start` and `stop` are **powers** of the base. The result runs from `base**start` to `base**stop` (included). The default base is 10.

SYNTAX

```
np.logspace(start, stop, num, base=10)
```

```
arr = np.logspace(0, 4, 6, base=2)   # 2**0 up to 2**4, 6 points

print(arr)
```

OUTPUT

```
[ 1.          1.74110113  3.03143313  5.27803164  9.18958684 16.        ]
```

9. Special Matrices for Linear Algebra (Bonus)

9.1 Diagonal matrix: np.diag

Puts your values on the main diagonal and 0 everywhere else.

SYNTAX
np.diag(v)

```
diagonal_matrix = np.diag([1, 2, 3, 4])

print(diagonal_matrix)
print(diagonal_matrix.shape)
```

OUTPUT
```
[[1 0 0 0]
 [0 2 0 0]
 [0 0 3 0]
 [0 0 0 4]]
(4, 4)
```

9.2 Identity matrix: np.eye

Items are 1 on the diagonal and 0 elsewhere. `N` is the number of rows, `M` the number of columns (default: same as `N`), and `k` shifts the diagonal (positive = to the right).

SYNTAX
np.eye(N, M=None, k=0)

```
identity_mat = np.eye(4)
print(identity_mat)
print(identity_mat.shape)

# np.eye(row, column, k)
mat = np.eye(3, 4)
print(mat)

mat = np.eye(3, 4, 1)          # diagonal moved one step to the right
print(mat)
```

OUTPUT
```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
(4, 4)
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]]
[[0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

Note: `np.eye` gives floats by default, which is why the output shows `1.` and `0.`.

10. Indexing and Slicing

10.1 Indexing

Counting starts at **0**. Use `arr[row, column]` to reach one item in a 2D array, and assign to it to change the value.

SYNTAX

```
arr1[index]
arr2[row, column]
```

```
print(arr1)
print(arr1[2])                  # third item

arr1[2] = 100                   # change the third item
print(arr1)
arr1[2] = 3                     # put the original value back

print(arr2)
arr2[0, 2] = 100                # row 0, column 2
print(arr2)
```

OUTPUT

```
[1 2 3 4 5]
3
[  1   2 100   4   5]
[[1 2 3]
 [4 5 6]]
[[  1   2 100]
 [  4   5   6]]
```

Note: `arr2[0][2]` (two steps) gives the same result as `arr2[0, 2]`, but `arr2[0, 2]` is the preferred style.

10.2 Slicing a 1D array

Syntax `arr[start:end:step]`. The start is included and the end is **not** included.

SYNTAX

```
arr[start:end:step]
```

```
print(arr1[1:4])                # items at index 1, 2, 3
print(arr1[::2])                # every second item

arr1_mod = arr1[1:4].copy()     # .copy() makes an independent copy
arr1_mod[2] = 200

print(arr1_mod)
print(arr1)                     # the original is unchanged
```

OUTPUT

```
[2 3 4]
[1 3 5]
[  2   3 200]
[1 2 3 4 5]
```

Note: Without `.copy()`, a slice is a **view** of the same data, so changing the slice also changes the original array.

```
demo = np.array([1, 2, 3, 4, 5])
view = demo[1:4]               # no .copy()
view[0] = 99

print(demo)                    # the original changed too
```

OUTPUT
```
[ 1 99  3  4  5]
```

10.3 Slicing a 2D array

Give one slice for the rows and one for the columns, separated by a comma. A single `:` means “everything”.

SYNTAX
```
arr[row_start:row_end:step, col_start:col_end:step]
```

```
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])   # re-created: arr2 was changed above

# getting a row
row_0 = arr2[0:1, :]
row_1 = arr2[1:, :]
print(row_0)
print(row_1)

# getting a column
col_0 = arr2[:, 0:1]
col_1 = arr2[:, 1:2]
print(col_0)
print(col_1)

# getting a portion (all rows, columns 0 and 1)
portion = arr2[:, 0:2]
print(portion)
```

OUTPUT
```
[[1 2 3]]
[[4 5 6]]
[[1]
 [4]]
[[2]
 [5]]
[[1 2]
 [4 5]]
```

Note: `::` and `:` both mean “the whole range”. The class notes wrote `arr2[:: , 0:1]` and `arr2[0:1 ,]`. These work the same as `arr2[:, 0:1]` and `arr2[0:1, :]`. Slices keep the 2D shape. `arr2[0]` (a plain index) gives a 1D row: `[1 2 3]`.

11. Advanced Indexing

11.1 Index lists (fancy indexing)

Give a **list of positions** to pick several items in any order. In 2D, two lists are read as pairs: (row, column).

SYNTAX

```
arr[[i, j, k]]
arr[[row1, row2], [col1, col2]]
```

```
lst = np.array([10, 20, 30, 40])
values = lst[[0, 3, 1]]        # positions 0, 3 and 1, in that order
print(values)

arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])   # re-created
print(arr2)
print(arr2[[0, 1], [1, 2]])    # items at (0, 1) and (1, 2)
```

OUTPUT

```
[10 40 20]
[[1 2 3]
 [4 5 6]]
[2 6]
```

Note: `arr2[[0, 1], [1, 2]]` does **not** select a 2 × 2 block. It picks only the pairs (row 0, column 1) and (row 1, column 2).

11.2 Boolean indexing

A condition gives True/False for every item. Using it inside `[ ]` keeps only the items where it is True. You can also use it to assign new values.

SYNTAX

```
arr[condition]
arr[condition] = value
```

```
print(arr2[arr2 > 1])          # items greater than 1
arr2[arr2 > 2] = 0             # replace items greater than 2 with 0

print(arr2)
```

OUTPUT

```
[2 3 4 5 6]
[[1 2 0]
 [0 0 0]]
```

Correction from class notes: The class code was `arr2[arr > 1]`, which uses a different array (`arr`) and fails or gives wrong results. The condition must be built from the **same** array: `arr2[arr2 > 1]`.

Note: Boolean indexing always returns a flat 1D array of the matching items.

12. Iterating over an Array

`np.nditer()` visits every item of an array, one by one, no matter how many dimensions it has.

SYNTAX
```
for item in np.nditer(arr):
```

```
arr = np.array([[10, 20, 30],
                [100, 200, 300]])

for i in np.nditer(arr):
    print(i)
```

OUTPUT
```
10
20
30
100
200
300
```

Correction from class notes: In the class notebook, `arr` was created but `np.nditer(arr3)` was used. Here the loop uses `arr`. With `arr3` it works the same way and prints all 18 items.

13. Reshaping and Flattening

13.1 reshape

Changes the shape without changing the data. The **total number of items must stay the same** (10 × 5 = 50 = 5 × 10).

SYNTAX
```
arr.reshape(rows, cols)
```

```
arr = np.random.randint(1, 100, size=(10, 5))   # sample - 10, feature - 5
print(arr.shape)

b = arr.reshape(5, 10)
print(b.shape)
print(b.ndim)
```

OUTPUT
```
(10, 5)
(5, 10)
2
```

Common mistake: The sizes must multiply to the same total. Reshaping 50 items into `(5, 9)` (45 items) raises a `ValueError`.

```
try:
    arr.reshape(5, 9)
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT
```
ValueError: cannot reshape array of size 50 into shape (5,9)
```

Note: A common layout for data is one **row per sample** and one **column per feature**.

13.2 flatten and ravel: 2D to 1D

Both turn an array into 1D. With `np.ravel`, `order='C'` reads row by row (default) and `order='F'` reads column by column.

SYNTAX

```
arr.flatten()
np.ravel(arr, order='C')   # 'C' = row by row, 'F' = column by column
```

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])      # small array so the result is easy to read

flatten = b.flatten()          # row by row
print(flatten)
print(flatten.ndim)

column_wise_flattening = np.ravel(b, order='F')   # column by column
print(column_wise_flattening)

row_wise_flattening = np.ravel(b, order='C')      # row by row
print(row_wise_flattening)
```

OUTPUT

```
[1 2 3 4 5 6]
1
[1 4 2 5 3 6]
[1 2 3 4 5 6]
```

Note: `flatten()` always returns a copy. `np.ravel()` returns a view when it can, so changing the result may change the original.

14. Joining, Transposing and Splitting

14.1 concatenate

Joins arrays. The arrays go inside a tuple. `axis=0` joins **row-wise** (one below the other). `axis=1` joins **column-wise** (side by side).

SYNTAX

```
np.concatenate((a, b), axis=0)
```

```
a = np.array([[6, 6, 3],
              [5, 6, 7]])
b = np.array([[26, 28, 21],
              [20, 26, 26]])   # fixed values (originally np.random.randint)

con_row = np.concatenate((a, b), axis=0)   # row wise
print(con_row)

con_col = np.concatenate((a, b), axis=1)   # column wise
print(con_col)
```

OUTPUT

```
[[ 6  6  3]
 [ 5  6  7]
 [26 28 21]
 [20 26 26]]
[[ 6  6  3 26 28 21]
 [ 5  6  7 20 26 26]]
```

Common mistake: For a row-wise join, the **number of columns** must match. For a column-wise join, the **number of rows** must match. Otherwise NumPy raises a `ValueError`.

```
x = np.array([[5, 5, 3],
              [6, 1, 2]])      # shape (2, 3)
y = np.array([[2, 8, 6]])      # shape (1, 3)

# Row wise: columns are the same (3 and 3), so no error
con_row = np.concatenate((x, y), axis=0)
print(con_row)

# Column wise: rows are not the same (2 and 1), so error
try:
    con_col = np.concatenate((x, y), axis=1)
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
[[5 5 3]
 [6 1 2]
 [2 8 6]]
ValueError: all the input array dimensions except for the concatenation axis must match
exactly, but along dimension 0, the array at index 0 has size 2 and the array at index 1 has
size 1
```

14.2 Transpose

Swaps rows and columns. Use the `.T` attribute.

SYNTAX

```
arr.T
```

```
mat = np.array([[10, 20, 30],
                [30, 40, 50]])

transpose = mat.T
print(transpose)
print(mat.shape, transpose.shape)
```

OUTPUT

```
[[10 30]
 [20 40]
 [30 50]]
(2, 3) (3, 2)
```

14.3 Splitting: array_split and split

`np.array_split` allows **unequal** parts. `np.split` needs the parts to be **equal**, otherwise it raises a `ValueError`.

SYNTAX
```
np.array_split(arr, n)
np.split(arr, n)
```

```
a = np.array([7, 3, 8, 2, 2, 5, 3, 3, 4, 1])   # fixed (was random)

# array_split -> unequal division is allowed
splitted_array = np.array_split(a, 3)
print(splitted_array)

# split -> must divide equally (10 items cannot split into 3 equal parts)
try:
    splitted_array = np.split(a, 3)
except ValueError as e:
    print("ValueError:", e)

# split works when the sizes are equal (9 items into 3 parts)
print(np.split(np.arange(1, 10), 3))
```

OUTPUT
```
[array([7, 3, 8, 2]), array([2, 5, 3]), array([3, 4, 1])]
ValueError: array split does not result in an equal division
[array([1, 2, 3]), array([4, 5, 6]), array([7, 8, 9])]
```

Note: When `array_split` cannot divide equally, the first parts get the extra items (sizes 4, 3, 3 above).

15. Arithmetic Operators and Mathematical Functions

15.1 Arithmetic operators

`+ - * / %` work **item by item** on arrays of the same shape. This is vectorization: faster than writing a loop.

```
x = np.array([10, 8, 30, 100])
y = np.array([2, 3, 4, 5])

add = x + y                    # same as np.add(x, y)
print(add)

sub = x - y
print(sub)

mul = x * y
print(mul)

div = x / y
print(div)

remainder = x % y
print(remainder)
```

OUTPUT
```
[ 12  11  34 105]
[ 8  5 26 95]
[ 20  24 120 500]
[ 5.          2.66666667  7.5        20.        ]
[0 2 2 0]
```

Note: `*` multiplies item by item. It is **not** matrix multiplication (see Section 21). `/` always gives floats.

15.2 Trigonometric functions

`np.sin`, `np.cos` and `np.tan` expect angles in **radians**. Use `np.deg2rad` to convert degrees to radians and `np.rad2deg` for the opposite.

SYNTAX

```
np.sin(x)   np.cos(x)   np.tan(x)
np.rad2deg(x)   np.deg2rad(x)
```

```
sin_val = np.sin(x)            # x is read as radians
print(sin_val)

cos_val = np.cos(x)
print(cos_val)

deg_con = np.rad2deg(x)        # radians to degrees
print(deg_con)

angles_deg = np.array([0, 90, 180])
print(np.deg2rad(angles_deg))  # degrees to radians
print(np.sin(np.deg2rad(90)))  # sin of 90 degrees
```

OUTPUT

```
[-0.54402111  0.98935825 -0.98803162 -0.50636564]
[-0.83907153 -0.14550003  0.15425145  0.86231887]
[ 572.95779513  458.3662361  1718.87338539 5729.57795131]
[0.         1.57079633 3.14159265]
1.0
```

Correction from class notes: The class code ended with an unfinished comment `# deg2rad`. A short `deg2rad` example is added above.

Common mistake: Passing degrees straight into `np.sin` gives wrong answers. Convert with `np.deg2rad` first.

15.3 Logarithms and square root

SYNTAX

```
np.log10(x)   np.log2(x)   np.sqrt(x)
```

```
x = np.array([10, 8, 16, 100])

base_10_log_val = np.log10(x)
print(base_10_log_val)

base_2_log_val = np.log2(x)
print(base_2_log_val)

sqrt_val = np.sqrt(x)
print(sqrt_val)
```

OUTPUT

```
[1.         0.90308999 1.20411998 2.        ]
[3.32192809 3.         4.         6.64385619]
[ 3.16227766  2.82842712  4.         10.        ]
```

Note: `np.log(x)` (no number) is the **natural** log (base e), not base 10. Variable names were tidied: `two_diye_bhag` became `base_2_log_val` and `sqrt` became `sqrt_val`.

15.4 Sum and cumulative (prefix) sum

`np.sum` adds all items. `np.cumsum` gives the running total after each item.

SYNTAX
```
np.sum(x)
np.cumsum(x)
```

```
s = np.sum(x)
print(s)

# prefix sum
cumul = np.cumsum(x)
print(cumul)
```

OUTPUT
```
134
[ 10  18  34 134]
```

Note: The class code used `np.cumulative_sum(x)`. It gives the same result but exists only in NumPy 2.1 or newer, while `np.cumsum` works in every version.

16. Logical Functions

Comparison operators compare item by item and return an array of True/False.

Operator	Meaning
`>` `<`	greater than, less than
`>=` `<=`	greater than or equal, less than or equal
`==` `!=`	equal to, not equal to

SYNTAX
```
np.all(condition)   # are ALL items True?
np.any(condition)   # is AT LEAST ONE item True?
```

```
x = np.array([10, 8, 16, 100])
y = np.array([2, 3, 16, 5])

greater_than = x > y
print(greater_than)

equal = x == y
print(equal)

print(np.all(greater_than))    # all True?
print(np.any(equal))           # any True?
```

OUTPUT
```
[ True  True False  True]
[False False  True False]
False
True
```

Common mistake: Use `==` to compare. A single `=` assigns a value and does not compare.

17. Sorting

17.1 In-place sort: arr.sort()

Sorts the **original** array itself and returns nothing.

```
x = np.array([10, 8, 16, 100])

z = x.copy()
print(z)

z.sort()                        # sorts z itself (in-place sort)
print(z)
```

OUTPUT

```
[ 10   8  16 100]
[  8  10  16 100]
```

Common mistake: `z = z.sort()` makes `z` equal to `None`, because `sort()` returns nothing. Just write `z.sort()`.

17.2 Copy sort: np.sort()

Returns a **new sorted array**. The original stays unchanged.

```
print(x)

sort_arr = np.sort(x)           # sorts a copy (copy sort)
print(sort_arr)

print(x)                        # x is unchanged
```

OUTPUT

```
[ 10   8  16 100]
[  8  10  16 100]
[ 10   8  16 100]
```

17.3 Sorting a 2D array

`axis=1` sorts **each row** (left to right). `axis=0` sorts **each column** (top to bottom).

SYNTAX

```
np.sort(arr, axis=1)
np.sort(arr, axis=0)
```

```
mat = np.array([[10, 3, 5],
                [8, 4, 9]])

hor_sort = np.sort(mat, axis=1)    # each row
print(hor_sort)

vert_sort = np.sort(mat, axis=0)   # each column
print(vert_sort)
```

OUTPUT

```
[[ 3  5 10]
 [ 4  8  9]]
[[ 8  3  5]
 [10  4  9]]
```

Note: `np.sort(mat)` with no `axis` sorts each row.

18. Searching

18.1 np.where(condition): find positions

Returns the **index positions** where the condition is True. The result is a tuple of arrays.

SYNTAX
np.where(condition)

```
x = np.array([10, 8, 16, 100])

index = np.where(x == 8)       # returns index
print(index)
```

OUTPUT
```
(array([1]),)
```

18.2 np.where(condition, x, y): replace values

Where the condition is True use `x`, otherwise use `y`. It returns a new array.

SYNTAX
np.where(condition, x, y)

```
arr = np.where(x > 8, x, 0)    # returns array
print(arr)

arr = np.where(x > 8, 30, 0)   # x and y can also be single values
print(arr)
```

OUTPUT
```
[ 10   0  16 100]
[30  0 30 30]
```

18.3 np.where on a 2D array

For 2D arrays, the position result has two arrays: the **row** numbers and the **column** numbers.

```
mat = np.array([[10, 3, 5],
                [8, 4, 9]])

index = np.where(mat > 8)      # returns index
print(index)

arr = np.where(mat > 8, mat, 0)    # returns array
print(arr)
```

OUTPUT
```
(array([0, 1]), array([0, 2]))
[[10  0  0]
 [ 0  0  9]]
```

Note: `(array([0, 1]), array([0, 2]))` means the True items are at (row 0, column 0) = 10 and (row 1, column 2) = 9.

18.4 Position of the maximum and minimum

`np.argmax` and `np.argmin` return the **index** (not the value). If the value appears more than once, the **first** index is returned.

SYNTAX

```
np.argmax(arr)
np.argmin(arr)
```

```
x = np.array([10, 100, 8, 16, 100])

maximum_value_index = np.argmax(x)   # returns 1st index
minimum_value_index = np.argmin(x)   # returns 1st index

print(maximum_value_index)
print(minimum_value_index)
```

OUTPUT

```
1
2
```

19. Counting in an ndarray

19.1 Counting items that match a condition

`np.count_nonzero(condition)` counts how many items are True.

SYNTAX

```
np.count_nonzero(condition)
```

```
a = np.array([5, 70, 65, 70, 5, 90, 65, 70, 61, 5])
# class version: a = np.random.randint(1, 100, size=(100,))

value_greater_than_60 = np.count_nonzero(a > 60)
print(value_greater_than_60)
```

OUTPUT

```
7
```

19.2 Unique values and how many times each appears

`np.unique` returns the sorted unique values. With `return_counts=True` it also returns how many times each one appears.

SYNTAX

```
np.unique(arr)
values, counts = np.unique(arr, return_counts=True)
```

```
unique_value = np.unique(a)
print(unique_value)
print("_____________________")

# return_counts=True gives a tuple: (unique values, counts)
unique_value = np.unique(a, return_counts=True)
print(unique_value)
print("_____________________")

# unpack into two arrays
unique_value, count = np.unique(a, return_counts=True)

print(unique_value)
print(count)
```

OUTPUT

```
[ 5 61 65 70 90]
_____________________
(array([ 5, 61, 65, 70, 90]), array([3, 1, 2, 3, 1]))
_____________________
[ 5 61 65 70 90]
[3 1 2 3 1]
```

Note: `count[i]` is the number of times `unique_value[i]` appears. Here, 5 appears 3 times and 70 appears 3 times. The class code used 100 random numbers, so a small fixed array is used here to make the result easy to check.

20. Statistical Functions

20.1 Loading data from a file: np.genfromtxt

`np.genfromtxt` reads numbers from a csv or txt file into an array. In a csv file the values are separated by commas, so `delimiter=','`. The first line holds column names, so `skip_header=1` skips it.

SYNTAX
```
np.genfromtxt('file.csv', delimiter=',', skip_header=1)
```

The file `student_scores.csv` was not included in the class notes. This setup code recreates it from the values printed in class. Skip it if you already have the file.

SETUP
```
csv_text = """Math,Subject_2,Subject_3
78,85,82
56,67,72
89,92,88
45,52,58
70,75,80
92,90,91
61,60,62
55,57,54
88,89,87
74,70,76
66,69,68
80,82,81
59,64,60
73,78,74
91,93,90
68,71,69
77,79,78
84,86,85
62,63,65
95,97,96
"""

with open('student_scores.csv', 'w') as f:
    f.write(csv_text)
```

```
data = np.genfromtxt('student_scores.csv', delimiter=',', skip_header=1)

print(data.shape)              # 20 students, 3 subjects
print(data[:3])                # first 3 rows only
```

OUTPUT
```
(20, 3)
[[78. 85. 82.]
 [56. 67. 72.]
 [89. 92. 88.]]
```

Note: `genfromtxt` returns floats, which is why the values print as `78.`. The file must be in the same folder as your notebook, or you must give the full path. The three column names above are placeholders, because the original header line was not in the notes.

20.2 Max, min, mean, median and standard deviation

Function	Meaning
`np.max` `np.min`	Largest and smallest value
`np.mean`	Average
`np.median`	Middle value when the data is sorted
`np.std`	Standard deviation: how spread out the values are around the mean. Low = values close to the average. High = values far from the average.

```
math_marks = data[:, :1]       # first column (all rows, column 0)

# .T shows the column as a row, which is easier to read
print(math_marks.T)

max_math_marks = np.max(math_marks)
print(max_math_marks)

min_math_marks = np.min(math_marks)
print(min_math_marks)

average_math_marks = np.mean(math_marks)
print(average_math_marks)

median_math_marks = np.median(math_marks)
print(median_math_marks)

std_math_marks = np.std(math_marks)
print(std_math_marks)
```

OUTPUT

```
[[78. 56. 89. 45. 70. 92. 61. 55. 88. 74. 66. 80. 59. 73. 91. 68. 77. 84.
  62. 95.]]
95.0
45.0
73.15
73.5
13.788672887555204
```

Note: `np.std` divides by N by default (population standard deviation). The class code wrote `data[::, :1]`; `data[:, :1]` is the same.

20.3 Correlation coefficient: np.corrcoef

Measures how two variables move together. The value is between -1 and +1: **+1** means they rise together, **-1** means one rises while the other falls, **0** means no linear relationship. Each **row** of the input is one variable.

SYNTAX

```
np.corrcoef(data)
```

```
study_hours = np.array([2, 4, 5, 7, 8])
exam_scores = np.array([65, 75, 78, 88, 92])

study_data = np.array([study_hours, exam_scores])   # one row per variable

correlation = np.corrcoef(study_data)

print("Correlation Matrix:\n", correlation)
```

OUTPUT

```
Correlation Matrix:
 [[1.         0.99859154]
 [0.99859154 1.        ]]
```

Note: The diagonal is 1 because each variable is perfectly correlated with itself. The value 0.9986 means study hours and exam scores have a very strong positive relationship. The variable was renamed from `data` to `study_data` so it does not overwrite the student scores array.

21. Linear Algebra

21.1 Matrix multiplication and trace

`np.dot(A, B)` multiplies two matrices. The **number of columns of A must equal the number of rows of B**. The result has the rows of A and the columns of B. `np.trace` adds the items on the main (primary) diagonal.

SYNTAX

```
np.dot(A, B)
np.trace(A)
```

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])      # shape (2, 3)

B = np.array([[7, 8],
              [9, 10],
              [11, 12]])       # shape (3, 2)

# Matrix multiplication: (2, 3) x (3, 2) gives (2, 2)
dot_product = np.dot(A, B)
print(dot_product)

# trace -> sum of the primary diagonal: 7 + 10
print(np.trace(B))
```

OUTPUT

```
[[ 58  64]
 [139 154]]
17
```

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(np.dot(A, B))
```

OUTPUT

```
[[19 22]
 [43 50]]
```

Note: Each result item is a row of A times a column of B. For example, the top-left item is 1×5 + 2×7 = 19. Do not use `A * B` for this: `*` multiplies item by item. Trace is normally used for square matrices; NumPy also accepts other shapes, as in class.

21.2 Determinant and rank

Both need a matrix of numbers. The determinant needs a **square** matrix (same number of rows and columns). The rank is the number of independent rows (or columns).

SYNTAX

```
np.linalg.det(M)
np.linalg.matrix_rank(M)
```

```
sq_mat = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

# Determinant of a matrix
det_of_sq = np.linalg.det(sq_mat)

# Rank of a matrix
rank_sq = np.linalg.matrix_rank(sq_mat)

print(det_of_sq)
print(rank_sq)
```

OUTPUT

```
0.0
2
```

Note: The rank is 2, which is less than 3, so the determinant is 0. On some computers the determinant prints as a tiny number such as `6.66e-16` instead of `0.0`. This is a floating-point rounding effect and means 0.